基于 FPGA 的多功能协议调试器

摘要

本设计基于国产高云 FPGA 平台，实现了一个高度集成、功能丰富且可灵活扩展的便携式电子系统调试与测试平台。

在数字信号处理方面，该系统集成了多种常用串行协议的转换与解析功能，包括 SPI、I2C、UART、多路 PWM 和 CAN，支持不定字节收发，能够作为 USB 转多协议设备与各类外设进行稳定通信。同时，系统具备数字信号测量能力，利用等精度测量结合频率预判分组机制，可在宽频率范围内实现高精度、快速的频率与占空比测量。在模拟信号处理方面，系统通过高速 ADC 实现 25Msps 的波形采集，并支持 Buffer 模式（基于 DDR3 存储）和流模式（基于千兆以太网）两种数据传输方式；配合 DAC 模块，可产生包括手绘波形在内的多种周期信号。设计的原创上位机软件提供协议控制、数字信号测量数据显示、模拟信号控制、时域波形显示、频谱分析及手绘波形量化等功能，实现了硬件控制与数据分析的一体化。测试结果表明，该系统各项功能运行稳定，可以同时使用，关键性能指标达到设计要求，为嵌入式系统与数字模拟电路的开发调试提供了“一站式”解决方案。

关键字：**串行协议转换 等精度测量法 模拟信号序列输出 DDR3 储存 以太网 波形显示**

第一部分　设计概述

1.1 设计目的

核心设计目的是基于国产高云 FPGA 平台，打造一个高度集成、功能强大且灵活可扩展的便携式电子系统调试与测试平台，为嵌入式系统、数字电路和模拟电路的设计开发人员提供一套“一站式”的硬件调试解决方案

1.2 应用领域

一、嵌入式系统开发与调试，作为 USB 转 SPI/I2C/UART/CAN 的设备，与各类设备进行通信。

二、作为数字电路分析仪，测量数字信号的频率、频率、占空比。

三、示波器：通过高速 ADC 实现模拟信号的采集、存储和波形显示，用于观察电路信号。

四、频谱分析：上位机的 FFT 功能，对采集的模拟信号进行频谱分析，用于检查电路中的噪声、谐波失真等。

五、信号源：产生各种标准波形或自定义波形，为模拟电路提供测试信号。

1.3 主要技术特点

一、协议转换支持 SPI/I2C/UART/CAN 多种常用协议，支持协议发送与接收

二、等精度法测量频率与占空比与分组测量，保证宽频率范围的测量精度与测量速度

三、原创上位机及 GUI 设计，可控制该模块的功能实现

四、上位机含示波器功能，可调节触发位置和触发电平

五、上位机可实现时域波形显示，频谱显示

六、支持串口+buffer 模式和以太网+流模式发送 ADC 数据至上位机

七、配合上位机可实现手绘波形输出

八、可自定义不同序列的输出波形

1.4 关键性能指标

一、协议转化与数字信号解析正常，可以与其他设备用对应协议通信

二、测量频率误差较小，低于 0.5%，低频情况下占空比误差低于 0.5%,高频时低于 3.5%

三、DAC 波形输出频率及形状符合设定（频率误差小于 5%），波形基本无毛刺，序列输出符合设定次序，手绘波形输出符合绘制

四、ADC 的串口+buffer 模式可对低频率波形进行采样，时域波形图符合输入，频谱图形状正确，其读数由受串口速率制约，并不可靠

五、ADC 的以太网流模式可对高频波形进行实时采样，时域波形图符合输入，频谱图形状正确，读数可靠

1.5 主要创新点

（一）除 CAN 发送为固定字节发送外，协议转化支持不定字节收发，提供更多选择

（二）采用粗略测量确定数字电路范围再精细测量的模式，保证测量精度与速度

（三）支持 DDR3 存储，串口发送的 buffer 模式

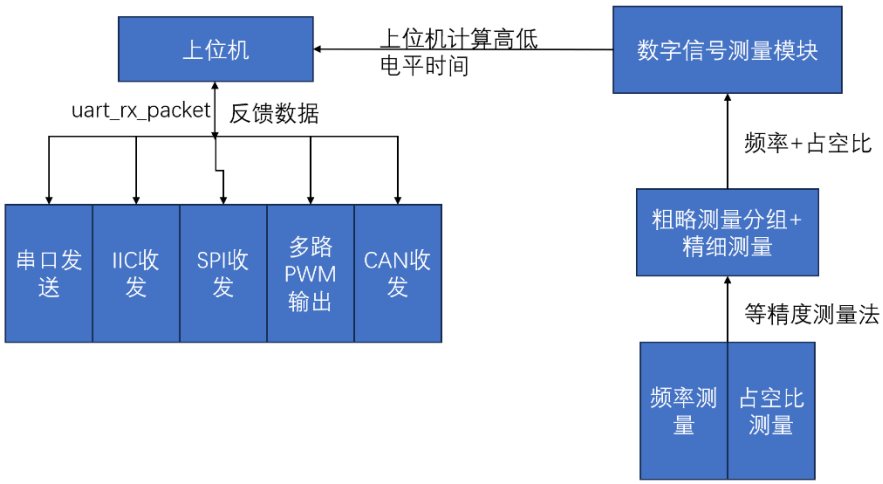
（四）支持以太网直接发送的流模式

（五）DAC 可对任意存储的周期波形数据进行发送，也可指定序列进行单通道发送和手绘波形发送

（六）上位机实现时域波形显示，频谱显示
（七）上位机实现对特定 ip 和端口进行监听
（八）上位机可将手绘波形进行量化

第二部分　系统组成及功能说明

2.1 整体介绍



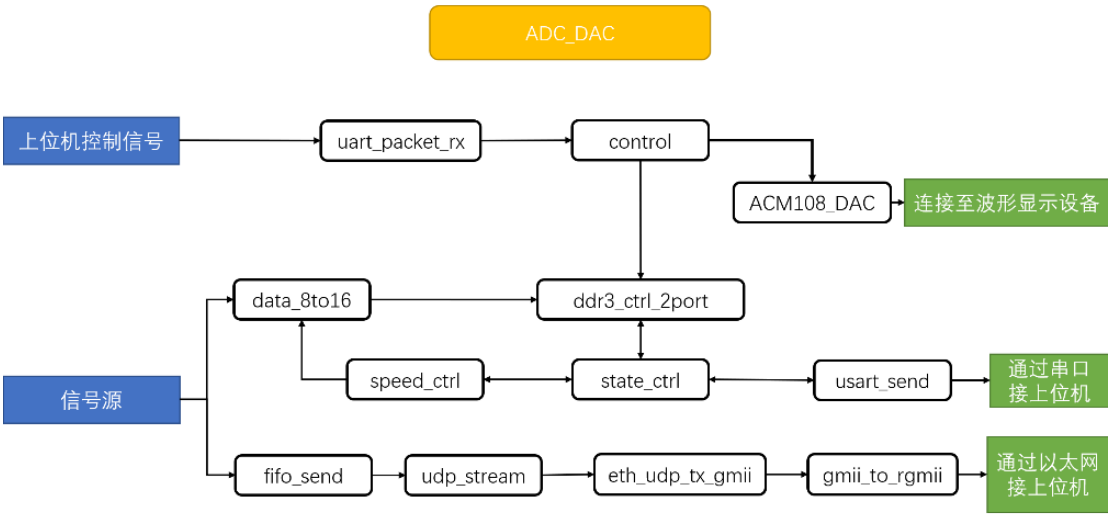


图 1 整体模块介绍

2.2 各模块介绍

（1）串行协议转化与解析部分

a.基于 CDC 的 USB 部分：

为节省 FPGA 资源，系统采用外置 FX2 芯片(CY7C68013)实现基于 CDC 的 USB 通信。FX2 配置为 Slave FIFO 模式，FPGA 通过读写其 fifo_data[7:0]接口进行 USB 数据收发。数据流经 FIFO 缓冲后，由 FX2_CDC_BYTE 模块处理单字节传输，最终由顶层模块 FX2_CDC_TOP 实现可变字节长度的

USB 数据收发。

b.UART 部分：

UART 模块实现了标准异步串行通信协议，支持全双工、可变字节长度传输及可编程波特率。模块采用分层设计：底层 uart_byte_tx 和 uart_byte_rx；上层模块 uart_data_tx 和 uart_packet_rx 实现不定包长数据包传输。

c.I2C 部分：

I2C 模块通过单一驱动文件 I2C_drive.v 实现了完整的 I2C 主设备功能，支持可编程通信速率和可变字节传输。模块核心为状态机，包含 W_DEVICE_ADDR, W_REG_ADDR, WDATA, R_DEVICE_ADDR, RDATA, STOP 六个状态。分频计数器产生 SCL 时钟：时钟低电平中点生成 wr_flag 用于数据赋值，高电平中点生成 rd_flag 用于数据采样。bit_cnt 计数器控制单次传输位数，byte_cnt 计数器管理多字节传输的字节数，共同协调寄存器地址和数据的收发流程。

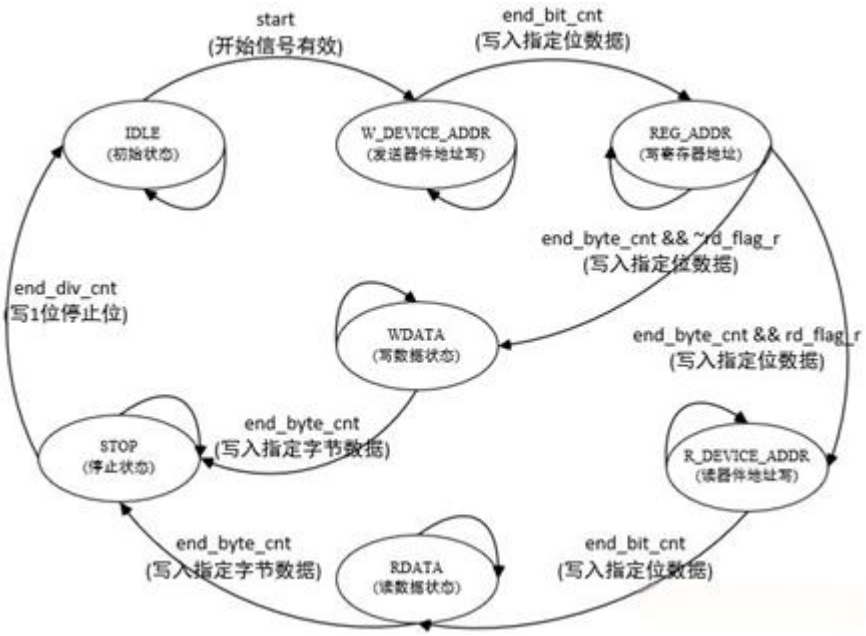


图 2 IIC 状态机转化流程

d.SPI 部分：

SPI 模块采用分层设计：底层 spi_master.v 实现核心驱动，支持多模式（三线、四线半双工、四线全双工）、可配置时钟极性/相位和数据端序的单字节传输；顶层 spi_top.v 通过复用底层模块实现可变字节传输。模块工作流程如下：使能后拉低片选激活从机，分频计数器生成 SPI 时钟。在指定时钟边沿按配置顺序移位输出数据，并采样输入数据存入接收移位寄存器。采用多次采样与多数判决机制确保可靠性。位计数器控制传输位数，支持背靠背连续传输。三线制下通过方向信号切换数据线方向。传输完成时输出完成标志和有效数据。

e.PWM 部分：

PWM 模块(PWM_drive.v)采用双计数器架构实现多路精确 PWM 输出：预分频计数器和自动重装计数器协同工作。当自动重装计数器达到预设阈值时执行电平切换，生成指定占空比和频率的波形。多路输出通过多次实例化

该模块实现。

f.CAN 部分：

系统实现了纯 Verilog 轻量级 CAN 控制器，其中底层模块 can_level_bit 通过按配置时序生成位时钟，检测采样边沿，处理位填充与空闲检测，为帧层提供采样/发送同步信号。帧层 can_level_packet 则按 CAN 帧格式解析/生成 ID、控制域、数据负载、CRC、ACK，并处理非破坏性仲裁。顶层模块负责提供上层接口，管理发送帧与远程帧的自动响应。控制器支持接收扩展帧及 1-8 字节数据负载。发送字节数可通过宏定义预设，确保收发适用性。

g.USB 控制模块

为统一管理各协议模块并保证数据稳定传输，定义了标准数据帧格式(见图 3)。上位机通过 USB 发送至 FPGA 的数据经解析后，路由至对应协议的顶层模块，实现协议收发（I2C 与 PWM 需额外数据帧转换顶层模块）。详细数据帧见附录 2。

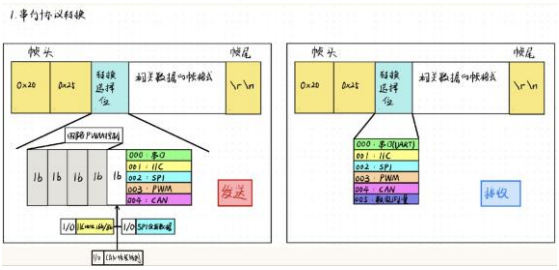


图 3 协议转化模块数据帧格式

（2）数字电路测量部分

数字信号测量模块采用等精度测量算法结合频率预判机制，在保证高精度的同时实现快速测量。频率测量与占空比测量独立分置于两个底层模块。频率测量模块使用同步闸门技术，在同步闸门时间内同时计数被测信号周期数和基准时钟周期数，通过比值计算频率，消除传统测频法与测周法的误差。占空比测量模块在闸门时间内分别计数高电平时钟数和总时钟数，通过比值获得精确占空比，在整个测量范围内保持恒定精度。两个模块均外接除法器，在保证计算精度与速度的同时显著降低资源消耗（逻辑单元从约 4200 降至约 760）。

为优化速度，顶层模块先用极低闸门时间粗略测量频率，再根据频率范围动态调整闸门时间，在保证精度的前提下提升测量速度。测量数据通过串口持续上传至上位机，数据格式与第一题一致，标识符为 3'd5，包含频率与占空比信息，高低电平时间由上位机计算。

（3）DAC 部分

ACM108_DAC：采用 DDS 的方式输出波形，DAC 时钟为 125Mhz，8 位精度输出，采用 DDS 的方式输出波形，相位累加器为 32 位，可通过串口配置频率控制字从而控制输出波形的频率。相位累加器根据不同的截取需求接入不同的存储模块。

正弦波，方波，三角波，锯齿波采用 11 位地址深度的 ROM。从相位累加器中截取高 11 位作为 ROM 的地址

手绘波形采用 9 位地址深度的双端口读写 RAM，写入端口接收上位机量

化后的波形数据，存储一周期波形，从相位累加器中截取 9 位作为 RAM 的读取端口的地址

uart_packet_rx:通过控制 uart_byte_rx 模块实现 20 字节数据包接收，每接收完数据包产生 uart_rx_done 信号

control:负责将数据包解包，uart_rx_done 每拉高一次，执行一次解包。并使用 fifo 将 200Mhz 系统的同步至 12.5Mhz，使控制信号稳定。将指令传递给其他模块。

（4）ADC 部分

buffer 模式

ddr3_ctrl_2port：ddr3 双端口模块，memory clock 配置为 400Mhz 时钟配比为 1：4，地址深度为 28 位，ddr3 每个地址存储 16 位数据，最大存储容量为 2^29 字节。使用 DDR3 芯片容量的，读写快的特点作为高速 ADC 采样的缓冲。通过 wrfifo 和 refifo 来对 ddr3 数据进行读写，消除跨时钟风险，并通过状态机控制实现了用简单的信号控制 ddr3 的读写。在 adc 采样，ddr3 缓冲，串口发送系统中，由于串口速度的制约，ddr3 的读写为非连续的，所以 ddr3 的状态控制需要严谨的状态机。工作流程为先将 adc 数据存储在 ddr3 中，存储数量达到设定容量后开始使用串口发送，发送顺序为先低八位，后高八位，每发送完成一个 16 位数据，便会拉高一次 rdfifo_rden，使输出计数器加一，从而确定何时能将 ddr3 存储的 adc 数据发送完毕，保证波形的稳定性。

data_8to16：由于 DDR3 内部以 16 位数据存储，这里每接收两个 8 位 DAC 数据便会通过两个时钟周期存储在 16 位变量中，每输出一个 16 位数据便产生一次使能信号。使数据与 ddr3 存储的每个地址对应的变量位数相同

speed_ctrl：接收 16 位的 adc 采样数据，可以控制 ADC 数据的采样率（设置多少个数据中选一个作为有效数据），每输出一个有效数据便产生一次使能信号。控制状态机进行写入操作。

state_ctrl：该模块是 ADC 采样数据经 DDR3 缓存后通过 UART 发送的核心控制器，通过状态机精确协调 FIFO 清零、采样使能、数据读写及 UART 发送时序，确保数据从 ADC 到 UART 的可靠传输。可将高速 ADC 数据缓存后进行串行输出。

usart_send：该模块负责发送 adc 数据，其配置波特率为 1000000，接收状态机的使能信号，每字节发送完成后便控制状态机继续发送。这里选择较高速率的波特率是为了尽量拉近 adc 采样与串口发送之间的速率差异

流模式

为了实现 25Mhz 采样率的 adc 数据的实时发送，采用高速的以太网通信，能将波形数据完整发送至上位机。实现实时波形观测，达到与示波器类似效果。Udp 数据帧的数据填充为 1472 个字节。具有较高的数据有效率

fifo_send:负责跨时钟频域的数据同步，并对数据进行计数。

udp_stream：udp 数据包的有效数据个数为 1472，，当 fifo 中的 Rnum 为 1472（实际应提前一个时钟信号），便执行一次发送（通过发送使能信号），并清零计数。此模块保障了 udp 数据帧的稳定发送，进而使波形稳定。

eth_udp_tx_gmii：负责按照 udp 协议填充数据帧，使其可以将填充好的数据帧发送至目标 ip 地址及端口号。数据帧使用了 crc 校验。产生的为 8 位 gmii 数据。

gmii_to_rgmii：ACG525 板载以太网芯片为 rgmii 接口，rgmii 接口有更少的管脚数量。使用高云 ODDR 原语件传输双倍数据速率信号，在发送总速率不变的条件下将 8 位信号转为 4 位信号，从而将 gmii 接口信号转换为 rgmii 接口信号，使其能被板载的以太网芯片发送出去

第三部分　完成情况及性能参数

串行协议转化与解析

UART 部分：

将串口与上位机进行连接，上位机通过 USB 发送指令后，可以看到数据能够顺利接收并传回

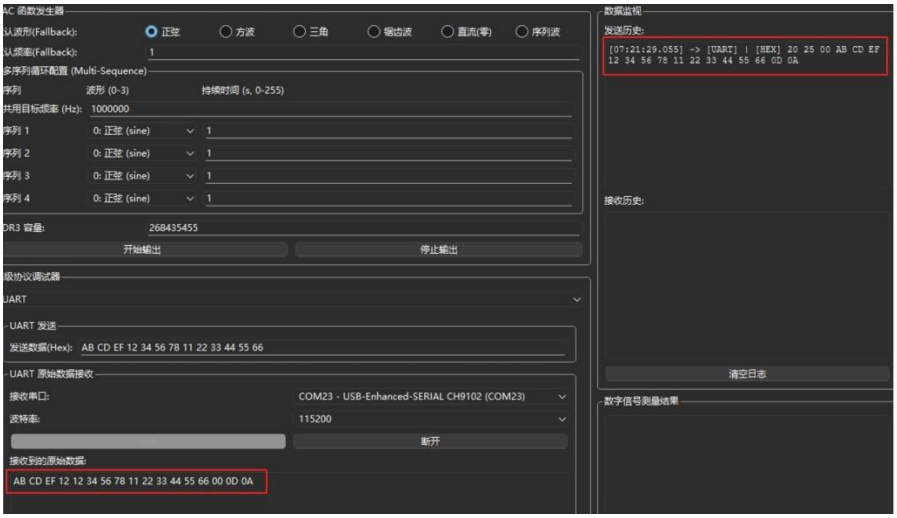


图 4 USB 转 UART

性能指标：

通信速率：取决于用户设置的波特率

最大包长：32 字节（可自定义）

资源占用：约占用 500 个寄存器与 750 个逻辑单元

I2C 部分：

通过与 MPU 9255 进行测试，首先我们向 MPU9255 发送支持旁路模式的指令，再发送读取 MPU9255 磁力计 ID 的命令，ID 读取成功，I2C 读写测试成功。（MPU9255 只有开启旁路模式才可使用磁力计）

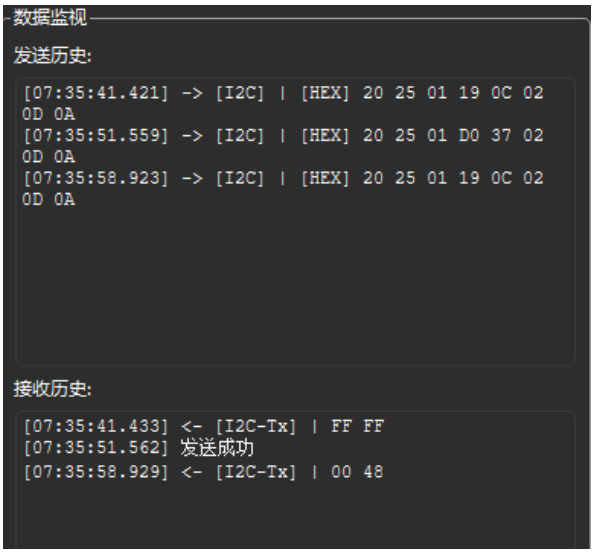


图 5 USB 转 I2C

性能指标：

支持速率：用户自定义（推荐与从设备保持一致）

地址支持：支持 8bits 与 16bits 寄在器地址

最大包长：16 字节（可自定义）

错误处理：无应答错误反馈

资源占用：约占用 450 个寄存器与 350 个逻辑单元

SPI 部分：

通过与 W25 Q 64（SPI 从设备）进行通信，发送 0X9F 读 ID 指令，接收数据为 EF 40 17,与实际相符， SPI 通信测试成功。

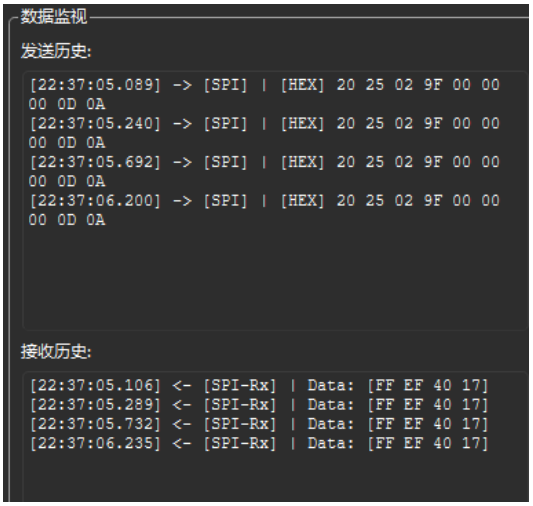


图 6 USB 转 SPI

性能指标：

通信速率：用户自定义

最大包长：16 字节（可自定义）

支持多种模式：可位意指定 CHOL 与 CPHA，可指定三种 SPI 模式（三线制、四线制半双工与四线制全双工）

姿源占用：约占用 400 个寄存器与 250 个逻辑单元

PWM 部分：

于上位机控制 PWM 波形，示波器显示波形正确、稳定，PWM 输出成功

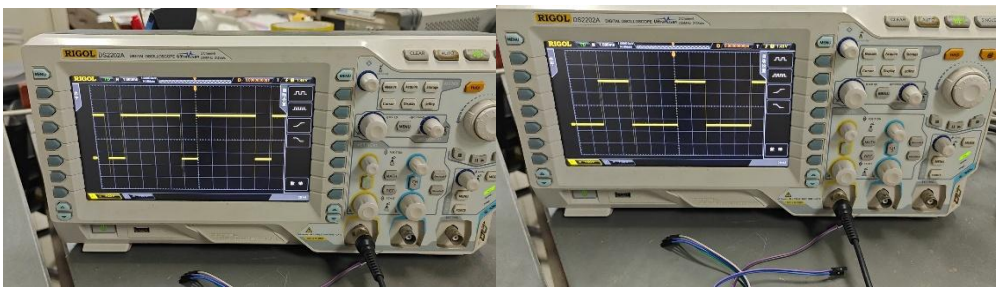


图 7 USB 转 PWM

性能指标:

最大输出路数：无上限，受引脚数量与 FPGA 资源限制，目前采用 4 路 PWM 输出

频率范围：最大频率取决于输入时钟频率（可采用锁相环增大频率），最低频率受计数器位宽影响。

占空比分辨率：将占空比映射为 0-255 的值，分辨率约为 3.9‰，可通过提高占空比位宽的方式改变占空比分辨率

资源占用：约占用 33*4 个寄存器与 56*4 个逻辑单元

CAN 部分：

与单片机连接，单片机成功接收数据，单片机发送数据，FPGA 成功接收数据，CAN 通信成功。（单片机实时显示 FPGA 发送的数据，按下按键可发送四个不断自增与一个固定的数据）

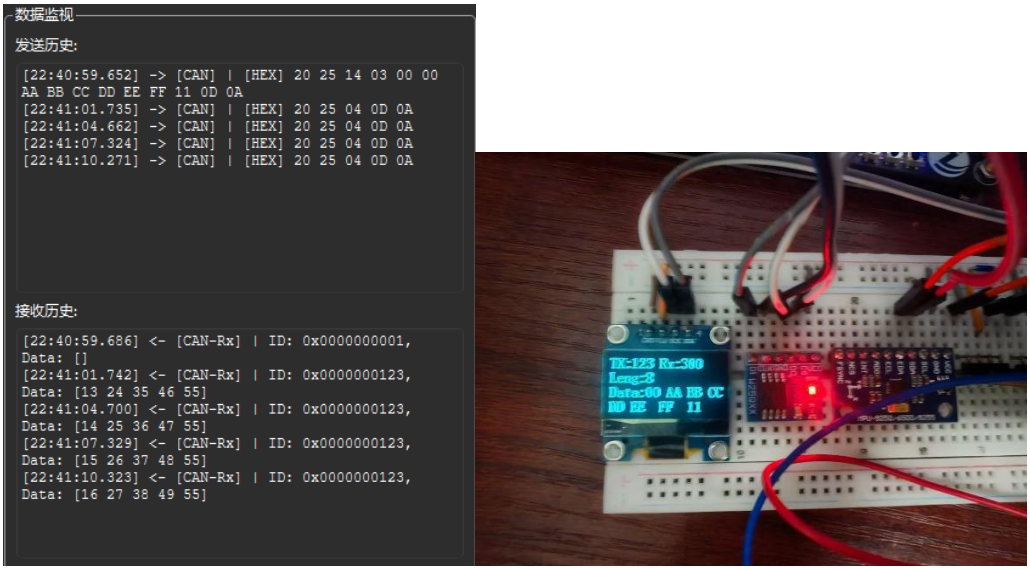


图 8 USB 转 CAN 部分

性能指标：

通信数据：用户自定义（支持高速模式与低速模式）

最大包长：8 字节收发

支持模式：接收部分支持标准帧与扩展帧，发送部分暂时只支持标准帧

资源占用：约占用 550 个寄存器与 620 个逻辑单元

数字电路测量部分：

通过信号发生器输出 2025HZ，53%占空比的方波，FPGA 计算后通过串口发送给上位机，计算值正确，数字电路测量验证成功。

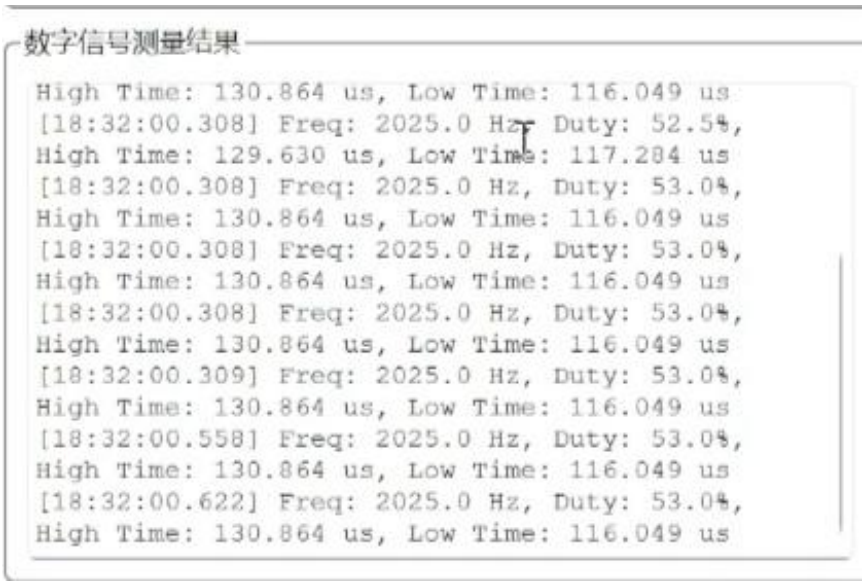


图 9 数字电路测量

性能指标：

测量范围：理论频率无上限，实际受 IO 口质量影响，最低频率受计数器位宽影响，实际测量过程中几十 HZ 到 10MHZ 均可测量

测量误差：受系统频率与闸门时间影响，目前设置下频率测量误差均低于 0.2%，占空比在低频测量时误差低于 0.2%，高频测量时相较实际占空比偏低，偏差 1-2%

测量时间：受闸门时间与待测频率影响，目前设置下除极低频（0-100HZ）测量时间大于 1s，中频与高频信号（1kHZ 以上）均低于 10ms

资源占用：约占用 550 个寄存器与 720 个逻辑单元

DAC 部分

DAC 可生成采样率为 125Mhz，采用 DDS 方式生成波形，相位累加器为 32 位。

手绘波形地址深度为 512 位，截取相位累加器高 9 位。低频输出波形完整

正弦波，方波，三角波，锯齿波地址深度为 2048，截取相位累加器高 11 位。支持高频输出，

示波器上可出现正确目标波形，如图

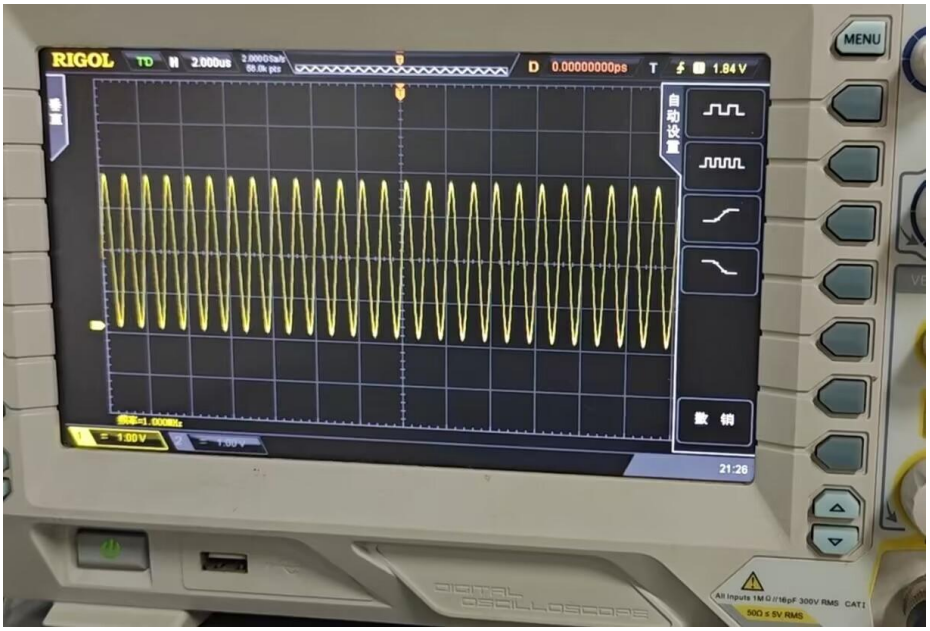


图 10 1MHZ 正弦波形图

正弦波实测频率数据：

目标输出 1.1Mhz，示波器测量为 1.087 Mhz ，误差为 1.2%

目标输出 1M hz，示波器测量为 1Mhz ，示波器与实际值一致

目标输出 90Khz，示波器测量为 90.56 Khz ，误差约为 0.6%

ADC 部分

DC 可实现 25Mhz 采样率

buffer 模式通过 ddr3 缓冲，1000000 波特率串口发送至上位机，低频测量时波形完整，测正弦波时 FFT 频谱图为垂直直线，由于发送速度受串口速度制约，FFT 频谱图无法读出正确频率

流模式通过以太网发送至上位机，高频时示波器功能正常，可调节触发电平和触发位置，FFT 频谱图为垂直直线，可通过 FFT 频谱图读出所测正弦波频率

实测数据：

Buffer 模式采集：信号源输出 88khz 正弦波，频谱图为垂直直线

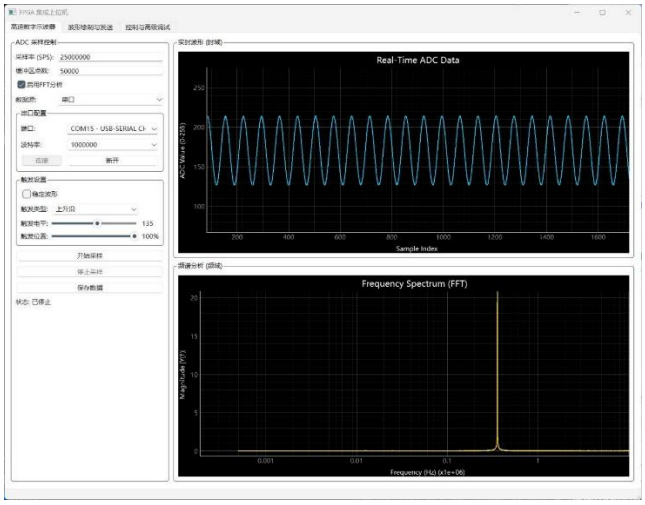


图 11 buffer 采样模式

流模式采集：信号源输出 1.2Mhz 正弦波，频谱图为垂直直线，FFT 频谱读出频率数据约为 1.2Mhz

流模式采集：信号源输出 0.8Mhz 正弦波，频谱图为垂直直线，FFT 频谱读出频率数据约为 0.8Mhz

流模式采集：信号源输出 1Mhz 正弦波，频谱图为垂直直线，FFT 频谱读出频率数据约为 1Mhz

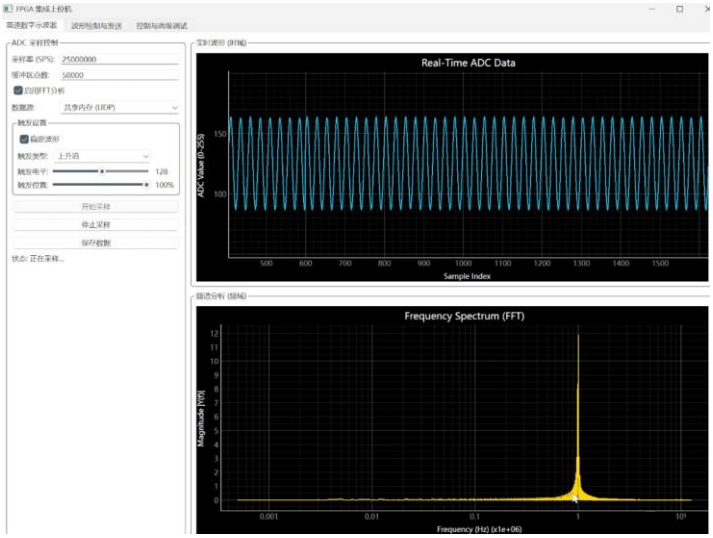


图 12 流模式采样（1Mhz）

第四部分 总结

4.1 可扩展之处

1. 可以增加对 USBHS 模式的支持，提高数据处理效率
2. 串行协议转换添加数字信号序列输出模式
3. 外接屏幕模块，实现离线模式
4. 增加对更多波形的预设

4.2 心得体会

在本项目的设计与实现过程中，我们深刻体会到将理论构想转化为稳定可靠产品的挑战与魅力。面对多协议集成、高速数据采集与实时传输等复杂需求，我们通过反复的模块调试与系统联调，不仅深化了对 FPGA 并行处理架构和数字信号处理原理的理解，更掌握了解决实际工程问题的系统方法。尤为可贵的是，团队协作贯穿始终，从硬件逻辑设计到上位机软件开发，成员间的紧密配合与知识互补是项目成功的关键。这段经历让我们认识到，一个优秀的作品不仅需要精湛的局部技术实现，更离不开对系统整体架构和用户体验的周全考量。

第五部分 参考文献

1. https://blog.csdn.net/weixin_50810761/article/details/136072803 **基于 FPGA 的 I2C 接口控制器（包含单字节和多字节读写）**

2. https://blog.csdn.net/Teminator_/article/details/141279117 SPI 通信协议详解

3. https://github.com/jakubcabal/spi-fpga SPI MASTER AND SLAVE FOR FPGA

4. https://blog.csdn.net/qq_36016038/article/details/124434179 CAN 通信学习笔记（一）

5. https://blog.csdn.net/wuzhikaidetb/article/details/112326945 FPGA 频率测量的三种方法（直接测量法，间接测量法，等精度测量法）

6. https://blog.csdn.net/Cauchy_Z/article/details/119148035 基于 FPGA+stm32 的等精度频率计

7. https://blog.csdn.net/cruisepenisula/article/details/149018125 二端口 DDR3 控制器模块

8. https://blog.csdn.net/woshiyuzhoushizhe/article/details/101159860 DDS 信号发生器原理与经典 DDS 信号发生器设计方案

9. https://blog.csdn.net/qq_31348733/article/details/95316094 基于 FPGA 的千兆以太网传输实例——ARP 和 UDP 的实现

10. https://blog.csdn.net/weixin_50810761/article/details/135898362 基于 FPGA 的 GMII 与 RGMII 接口相互转换

11. https://github.com/takeshineshiro/fpga_fibre_scan fpga_fibre_scan

第六部分　附录

1.整体资源占用：

Hierarchy

Update

Unit	File	Register	LUT	ALU	BSRAM	SSRAM	MULTALU27X18	MULT27X36
acg525_udp	**src\ACG525_UDP.v**	9037(0)	10319(3)	961(7)	33(0)	0 (0)	4(0)	2(0)
Gowin_PLL(your_instance_name)	src\gowin_pll\gowin_pll.v	100(0)	154(0)	18(0)	1(0)	0 (0)	0 (0)	0 (0)
Gowin_PLL_MOD(u_pll)	src\gowin_pll\gowin_pll_mod.v	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)
PLL_INIT(u_pll_init)	src\pll_init.v	100(100)	154(154)	18(18)	1(1)	0 (0)	0 (0)	0 (0)
ADC_DDR3_PLL(ADC_DDR3_PLL)	src\ADC_DDR3_PLL\ADC_DDR3_PLL.v	98(0)	151(0)	18(0)	1(0)	0 (0)	0 (0)	0 (0)
ADC_DDR3_PLL_MOD(u_pll)	src\ADC_DDR3_PLL\ADC_DDR3_PLL_mod.v	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)
PLL_INIT(u_pll_init)	src\pll_init.v	98(98)	151(151)	18(18)	1(1)	0 (0)	0 (0)	0 (0)
eth_pll(eth_pll)	src\eth_pll\eth_pll.v	99(0)	152(0)	18(0)	1(0)	0 (0)	0 (0)	0 (0)
eth_pll_MOD(u_pll)	src\eth_pll\eth_pll_mod.v	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)
PLL_INIT(u_pll_init)	src\pll_init.v	99(99)	152(152)	18(18)	1(1)	0 (0)	0 (0)	0 (0)
digital_top(digital_top_inst)	src\digital\digital_top.v	4237(695)	5887(1322)	644(0)	5(0)	0 (0)	4(0)	2(0)
FX2_CDC_TOP(FX2_CDC_TOP_inst)	src\digital\CDC2others\usb_cdc\FX2_CDC_TOP.v	874(818)	865(802)	63(31)	1(0)	0 (0)	0 (0)	0 (0)
FX2_CDC_BYTE(FX2_CDC_BYTE_inst)	src\digital\CDC2others\usb_cdc\FX2_CDC_BYTE.v	56(11)	63(18)	32(0)	1(0)	0 (0)	0 (0)	0 (0)
fifo_1024x8(tx_fifo_inst)	src\digital\CDC2others\usb_cdc\fifo_cdc.v	45(45)	45(45)	32(32)	1(1)	0 (0)	0 (0)	0 (0)
uart_data_tx(uart_data_tx_inst)	src\digital\CDC2others\uart_tx\uart_data_tx.v	312(280)	553(506)	6(6)	0 (0)	0 (0)	0 (0)	0 (0)
uart_byte_tx(u_uart_byte_tx)	src\digital\CDC2others\uart_tx\uart_byte_tx.v	32(32)	47(47)	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)
packet2IIC(packet2IIC_inst)	src\digital\CDC2others\I2C\packet2IIC.v	618(171)	1311(957)	16(0)	0 (0)	0 (0)	0 (0)	0 (0)
I2C_drive(I2C_drive_inst)	src\digital\CDC2others\I2C\I2C_drive.v	447(447)	354(354)	16(16)	0 (0)	0 (0)	0 (0)	0 (0)
spi_top(spi_top_inst)	src\digital\CDC2others\SPI\spi_top.v	334(278)	232(172)	8(8)	0 (0)	0 (0)	0 (0)	0 (0)
spi_modu(spi_modu_inst)	src\digital\CDC2others\SPI\spi_master.v	56(56)	60(60)	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)
packet2PWM(packet2PWM_inst)	src\digital\CDC2others\PWM\packet2PWM.v	299(167)	230(8)	192(0)	0 (0)	0 (0)	4(0)	0 (0)
pwm_drive(pwm_drive_inst_ch0)	src\digital\CDC2others\PWM\PWM_drive.v	33(33)	56(56)	48(48)	0 (0)	0 (0)	1(1)	0 (0)
pwm_drive(pwm_drive_inst_ch1)	src\digital\CDC2others\PWM\PWM_drive.v	33(33)	55(55)	48(48)	0 (0)	0 (0)	1(1)	0 (0)
pwm_drive(pwm_drive_inst_ch2)	src\digital\CDC2others\PWM\PWM_drive.v	33(33)	55(55)	48(48)	0 (0)	0 (0)	1(1)	0 (0)
pwm_drive(pwm_drive_inst_ch3)	src\digital\CDC2others\PWM\PWM_drive.v	33(33)	56(56)	48(48)	0 (0)	0 (0)	1(1)	0 (0)
can_top(can_top_inst)	src\digital\CDC2others\CAN\can_top.v	545(257)	624(101)	58(42)	4(4)	0 (0)	0 (0)	0 (0)
can_level_packet(u_can_level_packet)	src\digital\CDC2others\CAN\can_packet.v	288(243)	523(415)	16(0)	0 (0)	0 (0)	0 (0)	0 (0)
can_level_bit(u_can_level_bit)	src\digital\CDC2others\CAN\can_bit.v	45(45)	108(108)	16(16)	0 (0)	0 (0)	0 (0)	0 (0)
digital_measure(digital_measure_top_inst)	src\digital\digital_measure\digital_measure.v	560(62)	750(52)	301(9)	0 (0)	0 (0)	0 (0)	2(0)
digital_freq_measure(digital_freq_measure_inst)	src\digital\digital_measure\digital_freq_measure.v	252(114)	344(145)	142(0)	0 (0)	0 (0)	0 (0)	1(1)
divider(divider_inst)	src\digital\digital_measure\divider.v	138(138)	199(199)	142(142)	0 (0)	0 (0)	0 (0)	0 (0)
digital_duty_measure(digital_duty_measure_inst)	src\digital\digital_measure\digital_duty_measure.v	246(146)	354(169)	150(32)	0 (0)	0 (0)	0 (0)	1(1)
divider(divider_inst)	src\digital\digital_measure\divider.v	100(100)	185(185)	118(118)	0 (0)	0 (0)	0 (0)	0 (0)
ACM108_DAC(ACM108_DAC)	src\ADC_DAC_src\acm108_dac.v	119(32)	153(26)	32(0)	5(0)	0 (0)	0 (0)	0 (0)
DDS(DDS)	src\ADC_DAC_src\DDS.v	56(56)	49(49)	32(32)	5(0)	0 (0)	0 (0)	0 (0)
wave_ram(wave_ram)	src\wave_ram\wave_ram.v	0 (0)	0 (0)	0 (0)	1(1)	0 (0)	0 (0)	0 (0)
sine(sine_rom)	src\my_wave\sine\sine.v	0 (0)	0 (0)	0 (0)	1(1)	0 (0)	0 (0)	0 (0)
square(square_rom)	src\my_wave\square\square.v	0 (0)	0 (0)	0 (0)	1(1)	0 (0)	0 (0)	0 (0)
triangular(triangular_rom)	src\my_wave\triangular\triangular.v	0 (0)	0 (0)	0 (0)	1(1)	0 (0)	0 (0)	0 (0)
sawtooth(sawtooth_rom)	src\my_wave\sawtooth\sawtooth.v	0 (0)	0 (0)	0 (0)	1(1)	0 (0)	0 (0)	0 (0)
order_wave(order_wave)	src\ADC_DAC_src\order_wave.v	31(31)	78(78)	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)
usart_send(usart_send)	src\ADC_DAC_src\uart_send.v	73(42)	111(62)	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)
bin2bcd(bin2bcd)	src\ADC_DAC_src\DoubleDabble.v							
uart_tx(uart_tx_inst)	src\ADC_DAC_src\uart.v	31(31)	49(49)	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)
uart_packet_rx(uart_packet_rx_inst)	src\uart_packet_rx.v	150(117)	179(136)	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)
uart_byte_rx(uart_byte_rx_inst)	src\uart_byte_rx.v	33(33)	43(43)	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)

control(control)	src\ADC_DAC_src\control.v	32(0)	25(0)	4(0)	3(0)	0 (0)	0 (0)	0 (0)
fifo_uart(fifo_uart)	src\fifo_uart\fifo_uart.v	32(32)	25(25)	4(4)	3(3)	0 (0)	0 (0)	0 (0)
~fifo.fifo_uart(fifo_inst)	src\fifo_uart\fifo_uart.v							
data_8to16(data_8to16)	src\ADC_DAC_src\adc_8bit_to_16bit.v	26(26)	3(3)	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)
speed_ctrl(speed_ctrl)	src\ADC_DAC_src\speed_ctrl.v	1(1)	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)
state_ctrl(state_ctrl)	src\ADC_DAC_src\state_ctrl.v	107(107)	321(321)	31(31)	0 (0)	0 (0)	0 (0)	0 (0)
ddr3_ctrl_2port(ddr3_ctrl_2port)	src\ddr3_ctrl_2port\ddr3_ctrl_2port.v	3748(0)	2857(0)	165(0)	16(0)	0 (0)	0 (0)	0 (0)
fifo_ddr3_adapter(fifo_ddr3_adapter)	src\ddr3_ctrl_2port\fifo_ddr3_adapter.v	333(139)	689(324)	40(0)	8(0)	0 (0)	0 (0)	0 (0)
rd_data_fifo(rd_data_fifo)	src\rd_data_fifo\rd_data_fifo.v	97(97)	286(286)	20(20)	4(4)	0 (0)	0 (0)	0 (0)
~fifo.rd_data_fifo(fifo_inst)	src\rd_data_fifo\rd_data_fifo.v							
wr_data_fifo(wr_data_fifo)	src\wr_data_fifo\wr_data_fifo.v	97(97)	79(79)	20(20)	4(4)	0 (0)	0 (0)	0 (0)
~fifo.wr_data_fifo(fifo_inst)	src\wr_data_fifo\wr_data_fifo.v							
DDR3_Memory_Interface_Top(DDR3_Memory_Interfa...	src\ddr3_memory_interface\ddr3_memory_interface.v	3415(3415)	2168(2168)	125(125)	8(8)	0 (0)	0 (0)	0 (0)
~GW_DDR3_PHY_MC.DDR3_Memory_Interface_To...	src\ddr3_memory_interface\ddr3_memory_interface.v							
fifo_send(fifo_send)	src\fifo_send\fifo_send.v	112(112)	91(91)	24(24)	1(1)	0 (0)	0 (0)	0 (0)
~fifo.fifo_send(fifo_inst)	src\fifo_send\fifo_send.v							
udp_stream(udp_stream)	src\ADC_DAC_src\udp_stream.v	1(1)	4(4)	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)
eth_udp_tx_gmii(eth_udp_tx_gmii)	src\ADC_DAC_src\eth_udp_tx_gmii.v	134(102)	228(170)	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)
ip_checksum(ip_checksum)	src\ADC_DAC_src\ip_checksum.v							
crc32_d8(crc32_d8)	src\ADC_DAC_src\crc32_d8.v	32(32)	58(58)	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)
gmii_to_rgmii(gmii_to_rgmii)	src\ADC_DAC_src\gmii_to_rgmii.v	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)

图 13 整体工程资源消耗

2.协议转换数据帧

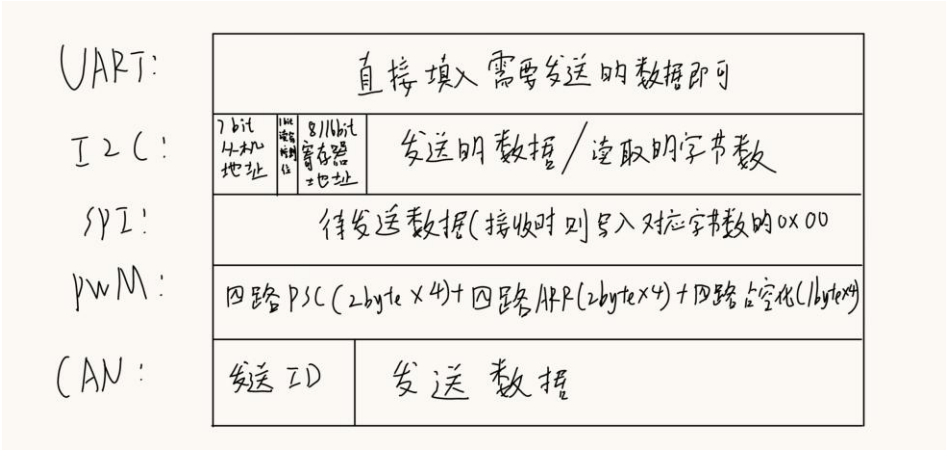


图 14 具体协议帧